FROM CONCEPT TO COMPUTATION

Identifying Group Appeals in Text

Alona Dolinsky & Dylan Paltra

COMPTEXT 2026 Workshop · University of Birmingham

Workshop Overview

PART 1

Concept & Manual Annotation

- Why study group appeals?
- Defining social groups & appeals
- Manual annotation codebook
- Reliability & validation

BREAK

~15 min

Break

PART 2

Computation & Practice

- NLP pipeline overview
- groupappeals Python package
- Hands-on exercise (~45 min)
- Discussion & wrap-up

PART 1

Concept & Manual Annotation

Defining group appeals ·
Building training data ·
Ensuring validity -> Intercoder Reliability

Why Study Group Appeals?

39%
of manifesto sentences contain a social group mention (PSoGA)

71.5%
of group appeals express a positive stance

304K
social groups detected across 791 manifestos, 8 countries

Why it matters:

- Group appeals shape electoral behaviour (Dassonneville et al. 2025; Thau 2021, 2024) and policy attitudes (Huber et al. 2024)
- Parties compete over groups, not just policies — comparative data have been scarce
- Few existing data rely on manual coding: single-country, single-group, short time-span, not publicly available data
- Automated methods enable large-scale, cross-national, longitudinal analysis*

Key Concept: Social Group

Definition: A segment of society or collection of people sharing common socio-demographic traits — ascriptive and/or acquired (sex, age, ethnicity, religion, residence, education, occupation, etc.) (Huber and Dolinsky 2023)

✓ INCLUDED

- Workers, elderly, women, students, farmers, religious communities
- Intersectional: 'young women', 'low-income families'
- Groupings within organisations: police officers, union members

✗ EXCLUDED

- Organised entities: trade unions, corporations, state authorities
- Political ideology/opinion/party groups: 'conservatives', 'activists', 'Republicans',
- Broad unspecific: 'everyone', 'all', 'people' (without modifier)

💡 *Key test: Does the group share socio-demographic traits (not just beliefs or political opinions)?*

Key Concept: Group Appeal

Definition: A statement in which a social group is explicitly mentioned and a stance (positive/negative/neutral) toward the group is expressed (Huber and Dolinsky 2023)

Group Detection	Stance	Policy Link (extra)
Is a social group **overtly** mentioned? Capture full span including modifiers.	Positive · Negative · Neutral Focus on stance toward the target, not overall sentiment.	Is a concrete policy (pledge, record, or criticism) directed at the group present?

Symbolic appeal: group mention without policy **Substantive appeal:** group mention + concrete policy

What Counts as an Overt Mention?

A group appeal requires an **explicit, named reference** to a social group. Implicit targeting — even when obvious — does not count.

✓ Overt mention — code this

"We will extend free childcare to all working mothers."

→ **Group:** *working mothers*
→ **Stance:** Positive
→ **Policy:** Yes

working mothers is explicitly named in the text — the group is in included in the data.

✗ No overt mention — do not code

"We will extend free childcare to 30 hours per week."

→ No social group is named in the text.
→ The policy clearly targets parents, but this is inferred, not stated.

We capture observable rhetoric, not inferred intentions. If the group is not in the text, it is not in the data.

Three Dimensions: Examples

*"…legislate to tackle hate crimes against **people with a disability**"* *(Fianna Fáil, 2020)*

Group: people with a disability

Positive | **Policy: Yes**

*"**The rich** are getting richer whilst everyone else struggles to make ends meet."* *(UK Greens, 2019)*

Group: the rich

Negative | **Policy: No**

*"The Conservative Party's policy of tax cuts has favoured **the rich** at the expense of **the struggling working class**."*

Group: the rich / the working class

Negative / Positive | **Policy: Yes / No**

43 Meaningful Group Categories

Selected examples — full list in Dolinsky et al. 2026 and handout

Demographic

- Children
- Young people
- Elderly people
- Women
- Men
- Adults

Economic

- Employees & workers
- Employers & business owners
- Lower class
- Upper class
- Middle class
- Farmers

Professional

- Healthcare professionals
- Education professionals
- Civil servants
- Law enforcement
- Sociocultural professionals
- White collar workers

Identity/Status

- LGBTQI
- Migrants & refugees
- Ethnic & national communities
- Religious communities
- People with disabilities
- Homeless people

Manual Annotation Codebook: Structure

- Unit of analysis: the natural sentence (linguistically coherent, replicable across languages)
- Three coding decisions per sentence:
- 1. Is a social group overtly mentioned and if yes, which one? (capture full span including all modifiers)
- 2. Stance toward the group: Positive / Negative / Neutral
- 3. Is a related policy present? Yes / No
- Multiple groups per sentence: each recorded separately with its own stance & policy label
- Intersectional groups (e.g., 'young women') assigned to multiple meaningful group categories
- 43 meaningful group categories — from Workers & Families to LGBTQI & Homeless People
- Residual 'Other' category for groups not fitting a specified category

Annotation: Common Challenges

General terms:	Code only if a modifier specifies a defined grouping: 'ordinary people', 'people in rural areas' ✓ — 'We will help people' ✗
Relational references:	'their families', 'their patients' in relation to a primary group: do NOT record as independent groups
Part of legislation / party name:	Group in a law title or party name (e.g. 'Workers' Party') → code as Neutral stance, no policy
Policy without overt group mention:	If the group is not explicitly named, do not code the appeal — even if the policy clearly targets them
Criticism of rival party's policy:	Record stance toward the group mentioned in the rival's policy (positive/negative), not toward the rival party

Building an Annotated Training Corpus

1	**Source selection:**	Which texts? Manifestos, speeches, social media, news — consider language(s)
2	**Sentence parsing:**	Split into natural sentences; handle lists and bullet points as standalone
3	**Annotation guidelines:**	Operationalise concepts with examples for each decision; pilot-test with coders
4	**Annotation platform:**	INCePTION, Doccano, Label Studio — assign & manage coder tasks
5	**Inter-rater reliability:**	Krippendorff's $\alpha \geq 0.8$ target; review disagreements, update guidelines, re-annotate + enhance training sample

Hands-On: Annotation Exercise

Working individually, annotate each sentence in Section 7 of the handout:

1	**Overt mention?**	Is a social group explicitly named? If yes — capture the exact span including all modifiers
2	**Stance**	What is the stance toward the group? Positive / Negative / Neutral
3	**Policy**	Is a related policy present? Yes / No
4	**Category**	Which meaningful group category does it belong to? (use Section 6 of the handout)

Then compare with your neighbour:

Where do you disagree?

- Which decision rule from the handout applies?

~10 min

BREAK

~15 minutes

Part 2 coming up: Computation & hands-on practice

PART 2

Computation & Practice

NLP pipeline ·
groupappeals Python package ·
Hands-on exercise

Reliability & Validation

PSoGA model performance (Krippendorff's α > 0.8 for all labels):

Token classifier (group detection)	Stance detection (NLI)	Policy detection (NLI)	Group category (multi-label)
F1: 0.957	**F1: 0.815**	**F1: 0.870**	**F1: 0.854**
P: 0.953 R: 0.961	P: 0.827 R: 0.807	P: 0.869 R: 0.874	P: 0.863 R: 0.844

Validation strategies:

- Face validity: keyness statistics by party family — do results match known ideological profiles? (yes)
- Replication: closely reproduces Thau (2019) trends for UK parties (workers, lower class, tenants, employers)
- Multi-language validation: additional annotation in Dutch, Spanish, and Danish

NLP Pipeline Overview

① Pre-processing

Parse texts → sentences
Create composite IDs
(party_date_sentenceID)

→

② Group Detection

mDeBERTaV3 Token Classifier
Identifies exact group spans

If group →

③a Stance Detection

NLI: Positive / Negative / Neutral

③b Policy Detection

NLI: Policy / No Policy

③c Group Classification

Multi-label: 43 categories

OUTPUT

Sentence-level dataset

rows · text, group, stance, policy, category

Manifesto-level dataset

Aggregate % appeals per category per manifesto

Key design choices & limitations:

- Unit of analysis = natural sentence (not quasi-sentence) — preserves group-stance link within a sentence
- Overt mentions only — implicit group targeting not captured; focus on observable rhetoric
- Currently 8 Western European countries / 5 languages — multilingual models, but validate for your context

Model Architecture: How It Works

- Token classifier (group detection): **mDeBERTaV3** fine-tuned on bilingual (EN+DE) annotated data — assigns LABEL_0/LABEL_1 to each token, then groups consecutive LABEL_1 tokens into entity spans
- Stance & policy detection: **Natural Language Inference** (NLI) approach — the model evaluates whether a hypothesis like 'The text is positive towards [group]' is true or false for each sentence; no re-training needed for new group categories
- Why NLI for stance/policy? Handles arbitrary group names without retraining; separates stance toward different groups within the same sentence; generalises across languages
- Group category classification: mDeBERTa multi-label classifier — each group token is assigned one or more of 43 categories simultaneously, enabling **intersectional** coding
- All models: **multilingual** mDeBERTaV3 base → fine-tuned on hand-coded English & German data → validated in Dutch, Spanish, and Danish

The groupappeals Python Package

Group Detection

extractgrouptoken.py

Token classifier identifies group spans in text; returns exact text, position, and confidence score

Stance Detection

stancedetection.py

NLI model assigns Positive / Negative / Neutral stance for each group mention

Policy Detection

policydetection.py

NLI model determines whether a policy directed at the group is present

Group Classification

classifymeaningfulgroups.py

Multi-label classifier assigns one or more of 43 meaningful group categories

Installation & Setup

```
# Install the package
pip install groupappeals

# Python import
from groupappeals import (extract_entities, detect_stance,
                          detect_policy, classify_groups,
                          run_full_pipeline)
```

⚠️ *Requires Python 3.8+ · PyTorch · transformers · GPU recommended for large corpora*

Input Data Format

party	year	sentence_id	text
Labour	2019	1	We will ensure that workers receive a real living wage.
Labour	2019	2	Families across the country deserve better public services.
Conservative	2019	1	We have created more jobs for young people than ever before.

```
# Create composite ID: party_year_sentence_id → 'Labour_2019_1', 'Labour_2019_2' …
```

Key requirements:

- CSV format; text column (default: 'text') + ID column (default: 'text_id') or composable columns
- Composite IDs (party + year + sentence_id) are strongly recommended — they maintain traceability throughout the full pipeline
- One sentence per row; remove headers, footnotes, and non-sentence content before running
- Language: models work best on EN, DE, NL, ES, DA — other European languages may degrade performance

Step 1: Group Detection

```
from groupappeals import process_csv

# From a CSV file (recommended)
df_groups = process_csv(
    input_file = 'my_texts.csv',
    text_column = 'text',
    id_column   = 'text_id',   # use your composite ID
    output_file = 'step1_groups.csv'
)

# Output columns: text_id, text, Entity, Average Score,
#                  Start, End, Exact.Group.Text
# text_id.0 = no group found; text_id.1/.2/... = each group
```

Each group mention in a sentence gets its own row. Rows with no group detected get suffix .0

Steps 2 & 3: Stance and Policy Detection

```
from groupappeals import process_stance_csv, process_policy_csv

# Stance detection — filter to rows WITH a group first
df_with_groups = df_groups[df_groups['Exact.Group.Text'].notna()]
df_with_groups.to_csv('groups_only.csv', index=False)

df_stance = process_stance_csv(
    input_file   = 'groups_only.csv',
    text_column  = 'text',
    group_column = 'Exact.Group.Text',
    clean_labels = True,        # adds 'Stance_Clean' column
    output_file  = 'step2_stance.csv'
)

df_policy = process_policy_csv(
    input_file   = 'step2_stance.csv',
    clean_labels = True,        # adds 'Policy_Clean' column
    output_file  = 'step3_policy.csv'
)
```

Step 4: Group Category Classification

```
from groupappeals import process_groups_csv

df_final = process_groups_csv(
    input_file    = 'step3_policy.csv',
    group_column  = 'Exact.Group.Text',
    score_threshold = 0.5,      # confidence threshold (0–1)
    split_groups  = True,       # → Group1, Group2, ... columns
    output_file   = 'final_results.csv'
)

# 'Meaningful Group' column contains list of assigned categories
# e.g. ['Employees And Workers', 'Lower Class']
# With split_groups=True: separate Group1, Group2 columns
```

Most group tokens (94%) receive at least one category; 8.3% fall into 'Other'

Full Pipeline: Everything in one step

```
from groupappeals import run_full_pipeline

# Python
result_df = run_full_pipeline(
    input_file         = 'my_texts.csv',
    output_file        = 'results_final.csv',
    text_column        = 'text',
    create_composite_id = ['party', 'year', 'sentence_id'],
    clean_labels       = True,
    split_groups       = True,
    batch_size         = 32
)

# CLI equivalent
groupappeals pipeline --input my_texts.csv --output results_final.csv \
    --create-composite-id party,year,sentence_id \
    --clean-labels --split-groups
```

Runs all four steps sequentially; intermediate files available; device auto-detected (CUDA → MPS → CPU)

Output: What You Get

`text_id`	Composite identifier (e.g. Labour_2019_1.2)
`text`	Original sentence text
`Exact.Group.Text`	Exact string of the detected group
`Stance`	Full NLI label ('The text is positive towards…')
`Stance_Clean`	'positive' / 'negative' / 'neutral'
`Stance_Confidence`	Model confidence score (0–1)
`Policy`	Full NLI label ('The text contains a policy…')
`Policy_Clean`	'policy' / 'no policy'
`Meaningful Group`	List of assigned category labels
`Group1, Group2…`	Individual category columns (if split_groups=True)

Validating Your Own Results

- Always **validate** before using results for inference — automated methods introduce errors
- Step 1 — Random sample check: draw ~100–200 sentences; verify group detection by hand; calculate precision & recall against your manual judgement
- Step 2 — Distribution checks: do stance proportions look plausible? (expect majority positive in manifestos); flag outlier parties or documents
- Step 3 — Convergent validity: if you have comparable manual data or existing datasets, compute correlations for overlapping categories
- Step 4 — Error analysis: examine false positives (things coded as groups that aren't) and false negatives (groups the model missed); update guidance if you plan to fine-tune
- Step 5 — quality_control=True flag: automatically checks that groups extracted by stance/policy models match original Exact.Group.Text (catches pipeline misalignments)

HANDS-ON EXERCISE

~45 minutes

A

Run the pipeline (20 min)

Apply groupappeals to the provided manifesto (or your own data). Run the full pipeline and inspect the output. Focus: does group detection match you manual annotations?

B

In the meantime: Manual Annotate (20 min)

Use the codebook to annotate the provided 20 sentences. Record: group detected? which group? stance? policy? Compare with your neighbour.

C

Explore & validate (15 min)

Compute a simple distribution: how many positive / negative / neutral stances? Check 5–10 outputs manually. What errors do you see? When does the model struggle? Then run the automatic performance metrics.

Adapting the Package to Your Research

- **Different text types**: the models were trained on manifestos — for speeches, social media, or news, validate carefully and consider fine-tuning on domain-specific data
- **Different languages**: mDeBERTaV3 is multilingual; validated on EN, DE, NL, ES, DA; test on your language before large-scale application
- **Fine-tuning**: collect a small annotated sample in your domain → use HuggingFace Trainer or SetFit to fine-tune the token classifier or NLI models
- **Extending categories**: the 43-category scheme is fixed in the current model; for domain-specific categories, train a new multi-label classifier on your annotations
- **HPC / large corpora**: use SLURM batch scripts; process in chunks; specify device='cuda'; batch_size=64+ on GPU
- R users: output is a CSV → read with read_csv() and analyse; package is Python-only

Wrap-Up & Discussion

- Group appeals are a distinct, measurable dimension of political communication — separate from sentiment and issue positions
- Manual annotation is the foundation: clear concepts + detailed codebook + inter-rater reliability checks
- Automated methods scale what manual coding cannot — but always validate, especially for new domains or languages
- The groupappeals pipeline gives you: group detection → stance → policy → category classification in one workflow

Discussion questions

- What annotation challenges did you encounter? How did you resolve disagreements?
- Where did the model perform well — and where did it struggle? What explains this?
- How might you adapt this approach for your own research context?

Thank you!

PSoGA Database: doi:10.7910/DVN/FVI3QW

groupappeals package: https://pypi.org/project/groupappeals/

Models on HuggingFace: https://huggingface.co/rwillh11

Contact: a.o.dolinsky@bham.ac.uk · dylan.paltra@uni-oldenburg.de

Workshop materials: https://github.com/dpltr22/comptext-2026-group-appeals